

For my lab 5, I tried to change the utility function of my agent. My original agent used a function that only used alpha beta pruning with the minimax algorithm with the utility being given from the number of squares we have and that the opponent had. But this time, I tried to make my agent compute “guaranteed squares” and give them more points. I scoped the agent out in my head for at least 10 hours and then spent some time coding it. The way I planned to do this, as I figured out after all of my thinking, was that some “shapes related to trapezoids” in the corner was guaranteed. These shapes are constructed by starting in a corner and making a rectangle in the corner, and then for one corner next to it imagining the sun shining from that corner and filling in the rectangle’s shadow. After some number of hours of thinking I was able to come up with the idea of these types of shapes in the corner, and later, I proved in my brain these would be guaranteed if one player had all squares of a shape of this type. The idea of “guaranteed sets of squares” was to make sure that after the pruning was complete it would still have a good heuristic function. My agent uses a value of 0 for empty squares and 1 for its own squares that aren’t guaranteed and a value of -1 for an opponent’s squares that aren’t guaranteed and a value of 3 for squares that are guaranteed to the agent and a value of -3 for squares guaranteed to the opponent. The purpose of my agent was to make sure that it would prioritize guaranteed squares instead of volatile squares. If I could do this again I would try to make a secondary class of “almost guaranteed” squares, which would be squares that would be guaranteed if only one empty square became filled with my squares. Then i’d try to keep a list of “last squares needed and try to secure them, which would

instantly make a large swath of squares guaranteed. However, doing this would take many more hours of coding if I tried it.